Vous travaillez pour une entreprise de création de jeux. Votre responsable vous a confié différentes tâches qui serviront à la mise en route d'un jeu sur le thème du "Seigneur des Anneaux".

DOSSIER 1	LES BATAILLES DE LA TERRE DU MILIEU	5 points

La première partie de votre travail consiste à construire la base de données qui va servir au premier volet du jeu : "Les batailles de la Terre du milieu".
Ce volet a pour objectif de plonger le joueur dans l'univers de l'Anneau et de lui permettre de faire des choix stratégiques lors des différentes attaques.
La base de données doit donc contenir tous les éléments nécessaires à la construction de la Terre du milieu.

Il existe plusieurs "pays" : le Rohan pays des Seigneurs des chevaux, le Gondor pays des Rois, le Mordor pays du Seigneur des ténèbres, la Comté pays des semi-Hommes, etc… Chaque pays possède un nom et une description.

Il existe plusieurs races : les Hobbits, les Hommes, les Nains, les Elfes, les Orcs, les Maia (Magiciens), etc…
Un pays est habité par une race et une race peut habiter dans plusieurs pays.

Chaque race dispose de personnages principaux : Sauron, Gandalf, Sarouman pour les Maia, Aragorn, Boromir, Faramir pour les Hommes, Frodon Sacquet, Bilbon Saquet, Gollum pour les Hobbits, Legolas, Galadriel pour les Elfes, Thorin, Balin pour les Nains.
Chaque personnage dispose éventuellement d'un titre (Roi, Seigneur, Capitaine…)

L'histoire se déroule dans différentes villes : chaque ville se trouve dans un pays précis (Edoras au Rohan, Minas Tirith au Gondor…).

Chaque bataille concerne une ville à une date précise et met en scène un certain nombre de représentants de différentes races.
Par exemple la bataille du gouffre de Helm comporte 6000 Hommes et 4000 Orcs.
Pour toutes les attaques, il faut savoir quels sont les personnages principaux présents et s'ils y meurent.

1.1	**Construire le modèle entité association ou le diagramme de classe correspondant aux informations du sujet.**

DOSSIER 2	PRIVACY BY DESIGN	5 points

Dans le cadre du RGPD (Règlement Général sur la Protection des Données), le concept de "Privacy by Design" (ou protection de la vie privée dès la conception) est l'un des principes clés. Ce principe vise à intégrer les considérations de protection des données dès la phase de conception d'un système ou d'un projet. Cela signifie que la protection des données personnelles doit être intégrée dès le départ, et non ajoutée après coup.

Le jeu sera proposé en ligne et les joueurs devront s'inscrire et s'abonner avec un paiement en ligne.

2.1	**En tant que développeur du jeu en ligne avec un système d'inscription et d'abonnement payant, indiquer les règles à respecter pour être conforme au RGPD, tout en minimisant les risques légaux et de sécurité.**
2.2	**Indiquer les informations que vous devez conserver lors de l'inscription et de l'abonnement et comment obtenir le consentement explicite des joueurs.**
2.3	**Quelles mesures de sécurité doivent être prises pour protéger les données des utilisateurs ?**
2.4	**Quels sont les droits des utilisateurs**
2.5	**Quelle durée de conservation des données faut-il respecter ?**

© Guy Verghote

DOSSIER 3	LES PERSONNAGES	5 points

La deuxième partie de votre travail consiste à mettre en place et à tester la classe Personnage qui sera utilisée dans le jeu.

La classe 'Personnage' est présentée en annexe 1.
Le programme de test unitaire est présenté en annexe 2
Le programme de test est présenté en annexe 3

3.1	**Indiquer l'intérêt d'avoir redéfini la méthode Equal dans la classe Personnage.**
3.2	**Indiquer l'intérêt d'avoir dérivé la classe Personnage de l'interface IComparable.**
3.3	**Compléter sur votre copie les lignes manquantes dans le programme de test unitaire (annexe 2). Vous indiquerez clairement avant chaque ligne la méthode correspondante.**
3.4	**Bien que vous ayez trouvé la bonne réponse sur le dernier test unitaire, le lancement de ce test unitaire échoue. Quelle en est la raison ?**
3.5	**Compléter sur votre copie les lignes manquantes dans le programme de test (annexe 3). Vous indiquerez clairement avant chaque ligne la méthode correspondante.**

DOSSIER 4	LES ANNEAUX MAGIQUES	5 points

La troisième partie du jeu porte sur "Les anneaux magiques".
Plusieurs anneaux magiques ont été créés et donnés à différents personnages importants : 9 rois humains, 3 Elfes, 7 Seigneurs Nains, et bien sûr l'Anneau unique de Sauron.
Chaque anneau possède ses propres pouvoirs. Par exemple, l'Anneau unique permet de disparaître lorsqu'on le met au doigt. Certains pouvoirs sont positifs, d'autres sont maléfiques.

Vous avez à votre disposition une base de données dont la description des relations correspondantes est donnée en annexe 4

4.1	**Présenter en langage SQL les requêtes qui permettent d'obtenir :** **a) Nom des anneaux et nom de leur propriétaire** **b) Nom des anneaux ayant des pouvoirs maléfiques** **c) Nombre de pouvoirs de chaque anneau.** **d) Nom et liste des pouvoirs de chaque anneau (une ligne par anneau)** **e) Nombre d'anneaux de chaque race**

Afin d'éviter de passer par une requête pour obtenir le nombre d'anneaux de chaque race, le champ nbAnneau est ajouté dans la table Race

```
Alter table Race Add nbAnneau int not null default 0
```

Vous êtes chargé de définir les triggers permettant de maintenir ce nouveau champ à jour. Le trigger permettant la mise à jour du champ nbAnneau après l'ajout d'un anneau est présenté dans l'annexe 5.

4.2	**Écrire le trigger permettant de mettre à jour le champ nbAnneau après la suppression d'un anneau**
4.3	**Écrire le trigger permettant d'interdire la modification du champ idProprietaire.**

Annexe 1 : Classe Personnage

```
namespace LesClasses {
   public class Personnage : IComparable<Personnage>    {

      // les attributs
      private string nom;
      private string race;
      private string? titre;

      // Constructeur
      public Personnage(string nom, string race)   {
         // à completer sur votre copie
      }

      public Personnage(string nom, string race, string titre)   {
         // à compléter sur votre copie
      }

      //propriétés
      public string Nom {
         get => nom;
         set => nom = value;
      }

      public string? Titre   {
         // à compléter sur votre copie
      }

      public string Race  {
         // à compléter sur votre copie
      }

      // redéfinition de la méthode ToString
      public override string ToString() => $"{nom,-20} {race,-10} {titre}";

      public override bool Equals(object o) {
         Personnage a = (Personnage)o;
        return string.Compare(nom, a.nom, CultureInfo.CurrentCulture, CompareOptions.IgnoreCase) == 0;
      }

      // surcharge des opérateurs pour s'appliquer sur des objets Etudiant
      public static bool operator ==(Personnage a1, Personnage a2)   {
         if (a1 is null)
            return a2 is null;
         else
            return a2 is null ? false : a1.Equals(a2);
      }
      public static bool operator !=(Personnage a1, Personnage a2)  {
         return !(a1 == a2);
      }

      public override int GetHashCode() { return this.ToString().GetHashCode(); }
```

```
    public int CompareTo(Personnage e)  {
        int res = race.CompareTo(e.race);
        if (res != 0) return res;
        res = nom.CompareTo(e.nom);
        return res;
      }
   }
}
```

Annexe 2 : Test unitaire

```
namespace TestUnitaire {
   [TestClass]
   public class TestPersonnage  {
      List<Personnage>? lesPersonages;

      [TestInitialize]
      public void InitialisationDesTests() {
         // création d'une collection de 20 personnages
         lesPersonages = new List<Personnage>() {
         new Personnage("Sauron","Maia", "Seigneur des Ténèbres"),
         new Personnage("Frodon Sacquet","Hobbit"),
         new Personnage("Gimli","Nain"),
         new Personnage("Thorin","Nain", "Roi sous la montagne"),
         new Personnage("Legolas","Elfe", "Prince de la Forêt Noire"),
         new Personnage("Thranduil","Else", "Roi"),
         new Personnage("Denethor","Homme", "Régent"),
         new Personnage("Bilbon Sacquet","Hobbit"),
         new Personnage("Gandalf","Maia"),
         new Personnage("Aragorn","Homme", "Roi"),
         new Personnage("Galadriel","Elfe", "Dame de Lothlórien"),
         new Personnage("Arwen","Elfe"),
         new Personnage("Elrond","Elfe", "Seigneur de Fondcombe"),
         new Personnage("Boromir","Homme", "Capitaine du Gondor"),
         new Personnage("Faramir","Homme", "Capitaine de la Tour de la Garde"),
         new Personnage("Sam Gamegie","Hobbit"),
         new Personnage("Pippin","Hobbit"),
         new Personnage("Merry","Hobbit"),
         new Personnage("Gollum","Hobbit"),
         new Personnage("Arathorn","Homme"),
         };
      }

      [TestMethod]
      public void TestMethodAdd() {
         Assert.AreEqual(20, lesPersonages.Count);
         // Ajouter le personnage Saroumane de race Maia à la collection
         à compléter sur votre copie
         Assert.AreEqual(21, lesPersonages.Count);
      }
```

```
    [TestMethod]
        public void TestMethodRemove()  {

            Assert.AreEqual(20, lesPersonages.Count);
            à compléter sur votre copie
            Assert.AreEqual(19, lesPersonages.Count);

            // supprimer le dernier personnage
            à compléter sur votre copie
            Assert.AreEqual(18, lesPersonages.Count);

            // supprimer tous les hobbits
            à compléter sur votre copie
            Assert.AreEqual(12, lesPersonages.Count);
        }

        [TestMethod]
        public void TestMethodFind()  {
            // rechercher le personnage Gandalf
            à compléter sur votre copie
            Assert.AreEqual("Gandalf", p.Nom);
            Assert.AreEqual("Maia", p.Race);
            Assert.AreEqual(null, p.Titre);

            // rechercher le personnage Elrond
            à compléter sur votre copie
        }

        [TestMethod]
        public void TestMethodFindAll()  {
            List<Personnage> lesHobbits;
            à compléter sur votre copie
            Assert.AreEqual(6, lesHobbits.Count);
        }

        [TestMethod]
        public void TestMethodWhere()  {
            Assert.IsNotNull(lesPersonages);
            // Recherche des personnages dont le nom commence par la lettre "A"
            à compléter sur votre copie
            Assert.AreEqual(4, lesPersonagesA.Count);
        }
    }
}
```

Annexe 3 : Extrait des méthodes à compléter du programme de test

La variable globale List<Personnage> lesPersonnages contient une collection de Personnage
La méthode afficher(lesPersonnages) permet d'afficher la liste des personnages

```
// tri sur le nom part la méthode sort
void triSurNom() {
    À compléter sur votre copie
    afficher(lesPersonnages);
}

// tri sur la race et le nom par la méthode Sort
void triSurRaceEtNom() {
    À compléter sur votre copie
   afficher(lesPersonnages);
}

// tri sur le nom par la méthode OrderBy
void triAvecOrderBy() {
     À compléter sur votre copie
}

// test des méthodes d'extension Linq
void nbPersonnageParRace() {
   /*
   Select race, count(*) as nb
   from lesPersonnages
   Group by race
   */

   // Grouper les personnages par race et compter le nombre de personnages dans chaque groupe
   À compléter sur votre copie

   // Afficher le nombre de personnages par race
   À compléter sur votre copie

   // Calculer la moyenne du nombre de personnages par race
   À compléter sur votre copie

   //Nom de la ou les races qui comportent le plus grand nombre de personnages
  // Calculer la valeur la plus grande puis parcourir en utilisant la méthode d'extension Where
  int nombreMax = lesRaces.Max(x => x.Nb);

  // Les races dont le nombre de personnages correspond à ce nombre
   foreach (var uneRace in lesRaces.Where(x => x.Nb == nombreMax)) {
      Console.Write("\t\t");
      Console.WriteLine(uneRace.Nom);
   }

    //Nom de la ou les races qui comportent le plus petit nombre de personnages
   À compléter sur votre copie

}
```

Annexe 4 : Le schéma relationnel

Anneau (id nom, description, idProprietaire)
Clé primaire : id
Clé étrangère : idPropriétaire en référence à id de la table personnage

Pouvoir (id, nom, malefique)
Clé primaire : id

PouvoirAnneau (idAnneau, idPouvoir)
Clé primaire : idAnneau, idPouvoir
Clé étrangère : idAnneau en référence à id de la table anneau
Clé étrangère : idPouvoir en référence à id de la table pouvoir

Race (id, nom)
Clé primaire : id

Personnage (id, nom, idRace)
Clé primaire : id
Clé étrangère : idRace en référence à id de la table Race

Annexe 5 : Trigger

Trigger permettant de mettre à jour le champ nbAnneau après l'ajout d'un nouvel anneau

```
create trigger apresAjoutAnneau after insert on Anneau
   for each row
   update race
   set nbAnneau = nbAnneau + 1
   where race.id = (select idRace from personnage where id = new.idProprietaire);
```

Trigger permettant de mettre à jour le champ nbAnneau après la suppression d'un anneau

À compléter sur votre copie (question (4.3)

Trigger permettant d'interdire la modification du propriétaire d'un anneau

À compléter sur votre copie (question 4.4)

Indication : Pour mettre fin à un trigger en retournant un message d'erreur :

signal sqlstate '45000' set message_text = '…';